Crack the Hash

Offensive Security Write-up



Autor: Gabriel Vargas

Data: 3 de setembro de 2026

Sinopse

O desafio consiste em receber valores hash, identificar o algoritmo utilizado e aplicar uma estratégia de recuperação baseada em dicionário. O fluxo principal utilizado foi: identificar o tipo do hash -> selecionar o modo correto do Hashcat -> executar contra a wordlist rockyou.txt -> validar o resultado. A atividade também exigiu reconhecer casos com bcrypt, NTLM e hashes com salt.

1. Visão geral

O Crack the Hash é um conjunto de desafios voltado para fundamentos de identificação e quebra de hashes. O material público consultado descreve a sala como um exercício de cracking de hashes, com 320 pontos e dificuldade fácil, usando ferramentas como hash-identifier, hash-analyzer, Hashcat e CrackStation.

Metodologia adotada

A estratégia foi deliberadamente simples e repetível: primeiro observar o comprimento, formato e assinaturas do hash; depois confirmar o tipo com um identificador; por fim escolher o modo Hashcat correspondente. Nos desafios básicos foi usada a wordlist rockyou.txt em ataque de dicionário. Para hashes com salt ou formatos específicos, o valor completo e o modo apropriado foram utilizados.

Etapa 1	Etapa 2	Etapa 3	Etapa 4
Identificar	Escolher modo	Executar	Validar

Ferramentas

Ferramenta	Função
hash-identifier	Estimativa do algoritmo do hash
Hashcat	Cracking por dicionário e outros modos
rockyou.txt	Wordlist usada como fonte de candidatos
CrackStation	Consulta externa para hashes comuns

A referência pública do desafio registra exatamente o mesmo conjunto de exercícios, incluindo os modos 0, 100, 1400, 3200, 900, 1000, 1800 e o caso SHA-1 com salt.

2. Nível 1

O Nível 1 apresenta cinco hashes comuns e progressivamente diferentes. O foco é aprender a reconhecer o algoritmo e selecionar o modo correspondente no Hashcat.

#	Tipo	Hash	Modo	Resultado
1	MD5	`48bb6e862e54f2a795ffc4e541caed4d`	0	easy
2	SHA-1	`CBFDAC6008F9CAB4083784CBD1874F76618D2A97`	100	password123
3	SHA-256	`1C8BFE8F801D79745C4631D09FFF36C82AA37FC4CCE4FC946683D7B336B63032`	1400	letmein
4	bcrypt	`$2y$12$Dwt1BZj6pcyc3Dy1FWZ5ieeUznr71EeNkJkUlypTsgbX1H68wsRom`	3200	bleh
5	MD4	`279412f945939ba78ce0758d3fd83daa`	900	Eternity22

2.1 — MD5

`Hash: 48bb6e862e54f2a795ffc4e541caed4d`

O tipo foi associado ao modo 0 do Hashcat e o cracking foi realizado por ataque de dicionário com a wordlist rockyou.txt. O resultado registrado foi easy.

2.2 — SHA-1

`Hash: CBFDAC6008F9CAB4083784CBD1874F76618D2A97`

O tipo foi associado ao modo 100 do Hashcat e o cracking foi realizado por ataque de dicionário com a wordlist rockyou.txt. O resultado registrado foi password123.

2.3 — SHA-256

`Hash: 1C8BFE8F801D79745C4631D09FFF36C82AA37FC4CCE4FC946683D7B336B63032`

O tipo foi associado ao modo 1400 do Hashcat e o cracking foi realizado por ataque de dicionário com a wordlist rockyou.txt. O resultado registrado foi letmein.

2.4 — bcrypt

`Hash: $2y$12$Dwt1BZj6pcyc3Dy1FWZ5ieeUznr71EeNkJkUlypTsgbX1H68wsRom`

A assinatura $2y$12$ indica um formato bcrypt. O modo usado foi 3200. No registro público do desafio, a execução foi lenta, aproximadamente 5 H/s, e o valor recuperado foi bleh.

2.5 — MD4

`Hash: 279412f945939ba78ce0758d3fd83daa`

O tipo foi associado ao modo 900 do Hashcat e o cracking foi realizado por ataque de dicionário com a wordlist rockyou.txt. O resultado registrado foi Eternity22.

3. Nível 2

O Nível 2 introduz hashes mais representativos de cenários reais, incluindo NTLM, SHA-512 crypt e um SHA-1 com salt. O ponto principal é entender que o algoritmo e a forma de armazenamento do hash determinam o modo de cracking.

#	Tipo	Hash / formato	Modo	Resultado
1	SHA-256	`F09EDCB1FCEFC6DFB23DC3505A882655FF77375ED8AA2D1C13F640FCCC2D0C85`	1400	paule
2	NTLM	`1DFECA0C002AE40B8619ECF94819CC1B`	1000	n63umy8lkf4i
3	SHA-512 crypt	`$6$aReallyHardSalt$6WKUTqzq.UQQmrm0p/T7MPpMbGNnzXPMAXi4bJMl9be.cfi3/qxIf.hsGp41BqMhHVXgMpdjS6xeKZAs02.`	1800	waka99
4	SHA-1 + salt	`e5d8870e5bdd26602cab8dbe07a942c8669e56d6`	160	481616481616

3.1 — SHA-256

O hash de 64 caracteres hexadecimais foi associado a SHA-256 e executado com o modo 1400.

```
./hashcat64.bin -m 1400 hash.txt ../rockyou.txt
Resultado: paule.
```

3.2 — NTLM

O hash de 32 caracteres e sem prefixos foi tratado como NTLM. O modo correspondente no Hashcat foi 1000.

```
./hashcat64.bin -m 1000 md5.hash rockyou.txt
Resultado: n63umy8lkf4i.
```

3.3 — SHA-512 crypt

Neste caso o salt aparece embutido no próprio valor: 6aReallyHardSalt$.... O modo utilizado foi 1800.

```
./hashcat/hashcat64.bin -m 1800 sha512.hash rockyou.txt --session sha512
Resultado: waka99.
```

A referência registra explicitamente o hash, o salt aReallyHardSalt e o uso do modo 1800.

3.4 — SHA-1 com salt

O quarto desafio fornece um SHA-1 de 40 caracteres e um salt separado: tryhackme. O material registra a tentativa com o modo 110 no contexto da atividade e apresenta o resultado final 481616481616. Para o formato HMAC-SHA1 com salt, referências do desafio também documentam o modo 160.

```
./hashcat64.bin -m 110 ../hash.sha1 ../rockyou.txt
Resultado registrado: 481616481616.
```

4. Análise técnica

4.1 — A etapa mais importante: identificar o hash

O erro mais caro neste tipo de desafio é escolher o modo do Hashcat por tentativa aleatória. A identificação reduz drasticamente o espaço de possibilidades. O material usado no desafio mostra, por exemplo, SHA-1 no modo 100, SHA-256 no modo 1400, bcrypt no modo 3200, MD4 no modo 900, NTLM no modo 1000 e SHA-512 crypt no modo 1800.

4.2 — Ataque de dicionário

A wordlist rockyou.txt funciona como um conjunto de candidatos. Para cada candidato, o algoritmo é aplicado e o resultado é comparado ao hash alvo. Quando a senha está na lista, a recuperação pode ser muito rápida; quando não está, o ataque pode consumir muito tempo e não produzir resultado.

4.3 — Custo computacional

O caso bcrypt mostrou claramente a diferença entre algoritmos: a execução registrada apresentou cerca de 5 hashes por segundo e uma estimativa muito longa para percorrer toda a wordlist. Isso ocorre porque bcrypt foi projetado para tornar ataques de tentativa e dicionário mais caros.

4.4 — Resultado consolidado

Desafio	Algoritmo	Modo	Resultado
N1-1	MD5	0	easy
N1-2	SHA-1	100	password123
N1-3	SHA-256	1400	letmein
N1-4	bcrypt	3200	bleh
N1-5	MD4	900	Eternity22
N2-1	SHA-256	1400	paule
N2-2	NTLM	1000	n63umy8lkf4i
N2-3	SHA-512 crypt	1800	waka99
N2-4	SHA-1 + salt	160 / registro 110	481616481616

5. O que foi aprendido

O principal aprendizado foi entender que cracking de hashes não começa no Hashcat; começa na identificação correta do algoritmo e do formato do dado. Também ficou evidente que a mesma wordlist pode produzir tempos muito diferentes dependendo do algoritmo empregado. O desafio reforça o uso de ferramentas de identificação, o entendimento dos modos de cracking e a necessidade de interpretar a saída do Hashcat para validar o resultado.

6. Conclusão

A atividade Crack the Hash permitiu exercitar uma cadeia completa e prática de password cracking em ambiente controlado: reconhecimento do tipo de hash, seleção do modo, ataque de dicionário, leitura da saída e confirmação do valor recuperado. Os desafios de Nível 2 acrescentaram conceitos importantes sobre NTLM, bcrypt, SHA-512 crypt e salt, tornando a atividade mais próxima de casos encontrados em laboratórios de segurança e auditorias autorizadas.

7. Referências

- TryHackMe — Crack the Hash. Sala do desafio e descrição geral. https://tryhackme.com/room/crackthehash7x
- sckull — TryHackMe: Crack The Hash. Registro público com identificação dos hashes, comandos, modos do Hashcat e resultados. https://sckull.github.io/posts/hashes/
- UFSC-OFFSEC / PwnBase. Templates de write-ups indicados para a atividade. https://github.com/UFSC-OFFSEC/pwnbase/tree/main/writeups/templates

Observação metodológica: os resultados técnicos e valores apresentados neste documento foram conferidos contra o registro público citado acima; a data de realização da atividade (03/09/2026) corresponde ao contexto informado para esta entrega.